Introduction

This project is about developing a Learning Management System (LMS) for a secondary school. The system will help the school manage users, courses, assignments, submissions, marks and feedback. It will have four main types of users: students, lecturers, Heads of Department and the system administrator.

The system will also make sure that marks are properly checked and moderated before they are released to students. The project will be developed using the Waterfall model, following the required software engineering processes from planning and requirements through to implementation, testing and maintenance.

1.Feasibility Study

The feasibility study was done to check if the proposed Learning Management System can be developed and used successfully within the given time and requirements.

1.1 Technical Feasibility

The system is technically feasible because it can be developed using the required tools such as C++17, SQLite, CMake and Git. SQLite can be used to store the school data locally, which also supports the requirement for the system to work offline. The system can also be designed to support the four different user roles and the required security features.

1.2 Operational Feasibility

The system is operationally feasible because it will make it easier for the school to manage courses, assignments, submissions and marks. Each user will only have access to the functions allowed for their role. The system will also be designed to be simple to use, especially for lecturers who may not have much IT experience.

1.3 Economic Feasibility

The system is economically feasible because it will use the specified development tools and SQLite for local data storage. This can reduce the need for expensive online services or additional software. The system can also reduce manual work involved in managing assignments, marks and student records.

1.4 Legal Feasibility

The system is legally feasible if student and assessment information is properly protected. Passwords must not be stored as plain text, and access to student information must be controlled according to each user's role. The system should also protect assessment records and personal information from unauthorised access.

1.5 Schedule Feasibility

The project is schedule feasible because the work is divided across seven weeks. Planning and requirements will be done first, followed by system design, implementation, testing, deployment and maintenance. Following the planned schedule will help make sure that enough time is given to each phase.

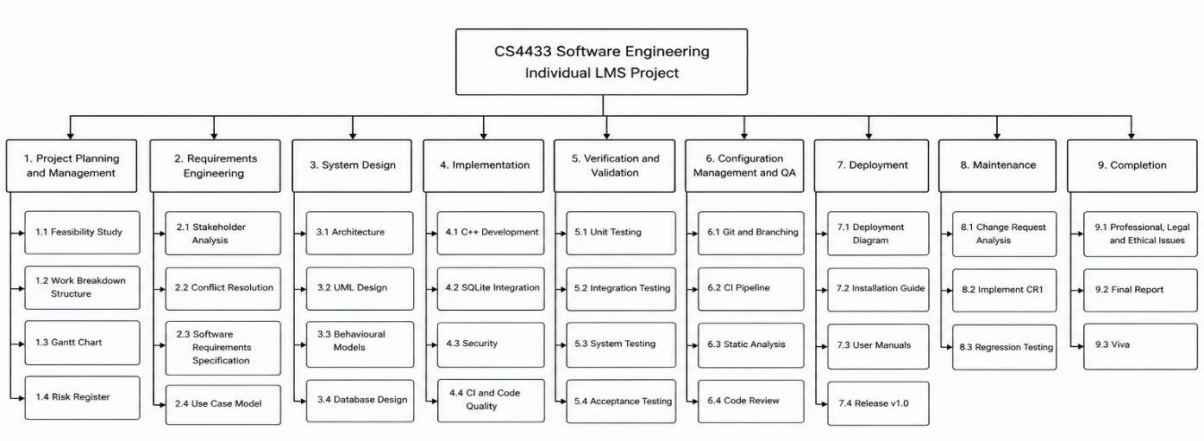
1.6 Recommendation

Based on the feasibility study, the proposed system is feasible to develop. The required technology is available, the system can meet the school's needs, and the project can be completed within the seven-week period if the work is properly planned and managed.

2. Work breakdown and schedule

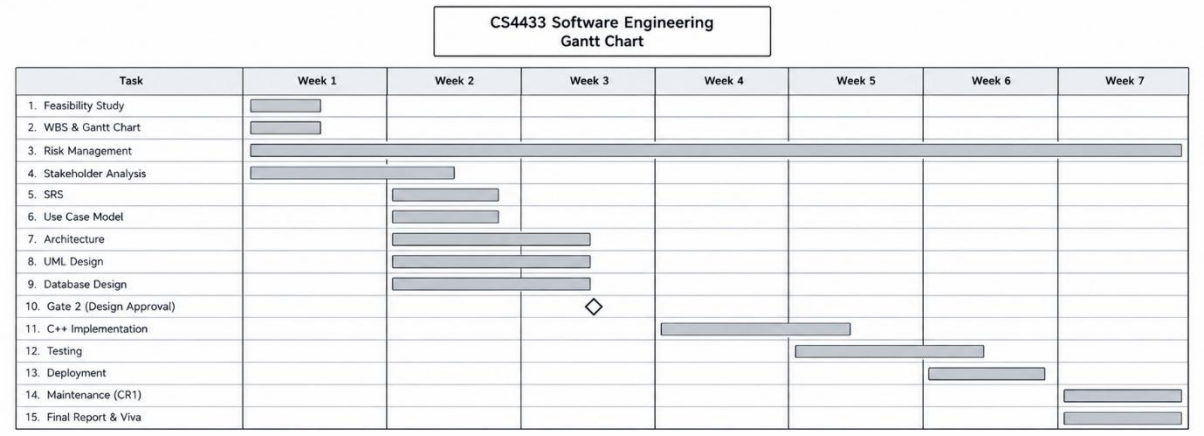
2.1 Work Breakdown Structure (WBS)

The WBS shows the main tasks of the project and how they are divided into smaller activities.



2.2 Gantt Chart

The Gantt chart shows when each task will be done during the seven weeks of the project, including the main dependencies and milestones.



2.2.1 Dependencies

1. Requirements Engineering → System Design
2. System Design → Gate 2 (Design Approval)
3. Gate 2 → Implementation
4. Implementation → Testing
5. Testing → Deployment
6. Deployment → Maintenance

2.2.2 Critical Path

Requirements Engineering → System Design → Gate 2 → Implementation → Testing → Deployment → Maintenance → Completion

3. Risk Register

The following risks were identified because they can affect the project, its quality or the time available to complete it. I'm using a 1–5 scale for likelihood and impact, where 1 is low and 5 is high and Exposure = Likelihood × Impact.

Risk	Likelihood	Impact	Exposure	Mitigation	Owner
Requirements are not clearly understood	3	5	15	Review the requirements carefully and clarify any conflicts or unclear points before design.	Requirements Analyst
The project takes longer than planned	4	5	20	Follow the weekly schedule and complete important tasks first.	Project Manager
Problems with C++ or SQLite during development	3	4	12	Test the technologies early and solve technical problems before implementation gets too far.	Developer
Important data may be lost	2	5	10	Use Git regularly and keep the project in a hosted repository.	Configuration Manager
Security problems such as weak password protection	3	5	15	Use password hashing, input validation and role-based access control.	Security Developer
Some requirements may be missed during development	3	4	12	Use the requirements traceability matrix to check that requirements are connected to the design, code and tests.	Requirements Analyst
Testing may not be completed properly because of limited time	3	4	12	Start preparing the test plan early and test important functions throughout development.	Test Engineer
Design and code may not match	3	4	12	Check the implementation against the UML diagrams and update the design when necessary.	System Designer